UNIVERSIDAD DE CALDAS

Módulo: Programación Web II

PARCIAL

Docente: Andrés Felipe Osorio Quintero

Unidad 2: Interactividad avanzada y almacenamiento local – Persistencia de datos en el navegador

Nombre del estudiante:	**Kevin Santiago Urán, Eliseth Nathaly Velasco Gómez, Juan Manuel Orozco y Mariana Diaz Jaramillo**
Fecha:	**5 de noviembre de 2026**

> ***Instrucciones***
> *Lea cada pregunta con atención antes de responder y base sus respuestas en los contenidos de la unidad.*
> *Sección I: marque una sola opción (A, B, C o D) por pregunta, encerrando la letra elegida.*
> *Secciones II y III: responda con argumentos claros y ejemplos concretos en el espacio dispuesto para cada pregunta.*
> *Escriba con letra legible y con tinta oscura.*

SECCIÓN I. SELECCIÓN MÚLTIPLE

Preguntas 1 a 20 · Selección múltiple con única respuesta

1. Un usuario elige el tema oscuro, el programa guarda esa decisión en una variable de JavaScript y, al recargar la página, la interfaz vuelve al tema claro. ¿Cuál es la explicación de este comportamiento?

A. El servidor borra la variable al recibir la nueva petición, porque HTTP no conserva el estado del cliente.

B. Las variables viven en la memoria de la página y se descartan al recargar, por lo que el código arranca con sus valores iniciales.

C. El navegador bloquea la escritura de variables cuando el usuario cambia una preferencia visual sin confirmarla.

D. Las variables solo conservan su valor tras una recarga si se declaran con `const` en lugar de `let`.

2. Una empresa debe decidir dónde guardar tres datos: (1) el identificador de una sesión autenticada, (2) el tema visual elegido por el usuario y (3) miles de registros para trabajar sin conexión. ¿Qué asignación es la más coherente con los criterios de la unidad?

- **A.** (1) `localStorage` con el token; (2) una cookie; (3) `sessionStorage` de la pestaña.
- **B.** (1) `sessionStorage` del navegador; (2) `IndexedDB`; (3) `localStorage` con arreglos.
- **C.** (1) `localStorage` con el token; (2) `sessionStorage`; (3) una cookie `HttpOnly`.
- **D.** (1) una cookie `HttpOnly` y `Secure`; (2) `localStorage`; (3) `IndexedDB`.

3. Una aplicación necesita que el servidor conozca la preferencia de idioma del usuario en cada petición, sin programar envíos manuales. ¿Qué mecanismo es el adecuado y por qué?

- **A.** Una cookie, porque el navegador la envía automáticamente al servidor en cada petición del dominio asociado.
- **B.** `localStorage`, porque el navegador adjunta sus entradas a cada petición HTTP dirigida al mismo origen.
- **C.** `sessionStorage`, porque comparte su contenido con el servidor mientras la pestaña permanezca abierta.
- **D.** `IndexedDB`, porque al ser asíncrona el servidor puede consultar su contenido sin bloquear el hilo.

4. ¿Cuál de las siguientes parejas de direcciones comparte el mismo `localStorage`?

- **A.** `http://localhost:3000` y `https://localhost:3000`
- **B.** `https://ejemplo.com` y `https://app.ejemplo.com`
- **C.** `https://ejemplo.com/inicio` y `https://ejemplo.com/contacto`
- **D.** `http://localhost:3000` y `http://localhost:8000`

5. Se ejecuta el siguiente código en una página servida desde un origen sin datos previos. ¿Qué imprime la consola y por qué?

```
localStorage.setItem("cantidad", 5);
const n = localStorage.getItem("cantidad");
console.log(n + 1);
```

- **A.** `6`, porque `setItem()` conserva el tipo numérico del valor recibido.
- **B.** `51`, porque el valor se guardó como el texto `"5"` y el operador `+` concatena.
- **C.** `6`, porque `getItem()` convierte automáticamente a número lo que recupera.
- **D.** `null`, porque `setItem()` no admite números como valor, solo cadenas.

6. En la primera visita de un usuario se ejecuta `localStorage.getItem("pwc2:tema")` y esa clave nunca se había guardado. ¿Qué devuelve y qué implica para el código?

- **A.** Devuelve `null`; el código debe distinguir «no hay dato» de un dato vacío y asignar un valor por defecto.
- **B.** Devuelve `undefined`; el código debe compararlo con `typeof` antes de aplicar cualquier valor.
- **C.** Devuelve una cadena vacía; el código puede usarla directamente como si fuera un tema válido.
- **D.** Lanza una excepción `SecurityError`; el código debe capturarla para poder continuar con la carga.

7. Una página incrementa dos contadores en cada carga: uno en `localStorage` y otro en `sessionStorage`. En la pestaña 1 ambos valen 4. El usuario abre la misma dirección en una pestaña nueva (no duplicada). ¿Qué valores muestra esa pestaña tras cargar?

- **A.** `localStorage`: 1; `sessionStorage`: 1
- **B.** `localStorage`: 5; `sessionStorage`: 5
- **C.** `localStorage`: 1; `sessionStorage`: 5
- **D.** `localStorage`: 5; `sessionStorage`: 1

8. Dos pestañas del mismo origen muestran una lista de tareas guardada en `localStorage`. Se quiere que un cambio hecho en una pestaña aparezca en la otra sin recargar. ¿Cuál afirmación describe correctamente el evento `storage`?

- **A.** Se dispara en la misma pestaña que ejecutó `setItem()`, inmediatamente después de cada escritura.
- **B.** Se dispara en todas las pestañas del origen, incluso cuando se reasigna exactamente el mismo valor.
- **C.** Se dispara en las demás pestañas y ventanas del mismo origen, y solo cuando el valor realmente cambia.
- **D.** Se dispara únicamente con `sessionStorage`, entre pestañas que fueron duplicadas a partir de la original.

9. En la función `almacenamientoDisponible()` del documento, incluso la lectura de `window[tipo]` se ubica dentro del bloque `try`. ¿Cuál es la razón?

- **A.** Porque `window[tipo]` es una operación asíncrona que devuelve una promesa.
- **B.** Porque en algunos navegadores esa simple lectura puede lanzar `SecurityError` si el acceso está bloqueado.
- **C.** Porque la lectura de `window[tipo]` agota la cuota del origen y provoca un `QuotaExceededError`.
- **D.** Porque `removeItem()` lanza un error cuando la clave de prueba todavía no existe en el almacén.

10. Una aplicación guarda en `localStorage` un arreglo cada vez más grande y, en algún momento, `setItem()` lanza `QuotaExceededError`. ¿Qué respuesta es coherente con la programación defensiva descrita en la unidad?

- **A.** Capturar la excepción con `try/catch`, informar el fallo si afecta al usuario y seguir funcionando sin persistencia.
- **B.** Pasar los datos a `sessionStorage`, que según la unidad no tiene una cuota asociada al origen.
- **C.** Repetir la llamada a `setItem()` dentro de un bucle hasta que el navegador acepte la escritura.
- **D.** Mover la escritura a un Web Worker para que no bloquee el hilo principal ni agote la cuota.

11. Una aplicación guarda en `localStorage` estructuras de varios megabytes en cada pulsación de tecla y la interfaz se congela por momentos. ¿Cuál es la causa y la mitigación más adecuada según la unidad?

- **A.** Las escrituras son asíncronas y se acumulan en cola; conviene usar `await` antes de cada `setItem()`.
- **B.** `JSON.stringify()` corre en un hilo aparte que se satura; conviene reemplazarlo por `structuredClone()`.
- **C.** El evento `storage` se dispara en bucle en la misma pestaña; conviene eliminar el oyente del evento.
- **D.** Cada lectura y escritura bloquea el hilo principal; conviene guardar menos datos, agrupar escrituras o usar `IndexedDB`.

12. Se ejecuta el siguiente código. ¿Cuál es el texto que imprime la consola?

```
const datos = { a: undefined, b: NaN, c: [undefined, 3], d: new Set([1]) };
console.log(JSON.stringify(datos));
```

- **A.** `{"b":null,"c":[null,3],"d":[1]}`
- **B.** `{"a":null,"b":null,"c":[null,3],"d":{}}`
- **C.** `{"b":null,"c":[null,3],"d":{}}`
- **D.** `{"b":"NaN","c":[3],"d":{}}`

13. Una aplicación guarda tareas como `{ titulo, vence: new Date() }` con `JSON.stringify()`. Al leerlas con `JSON.parse()`, la llamada `tarea.vence.getFullYear()` lanza un `TypeError`. ¿Cuál es la causa y la solución adecuada?

- **A.** `Date` se serializa como cadena ISO 8601 y `parse()` la devuelve como cadena; se debe reconstruir el `Date` con un `reviver`.
- **B.** `stringify()` descarta las propiedades de tipo `Date`; se debe evitar esa pérdida usando un `replacer` al guardar.
- **C.** `Date` lanza un error al serializarse; se debe convertir la fecha a `BigInt` antes de guardarla.
- **D.** `parse()` convierte las fechas en números de milisegundos; se debe aplicar `Number()` al leerlas.

14. Un usuario edita manualmente en las herramientas de desarrollo el valor de una clave y deja `{nombre: 'Ana'}`. La aplicación lo lee con el módulo `leerJSON()` del documento. ¿Qué sucede?

- **A.** `JSON.parse()` devuelve el objeto con `nombre` igual a `'Ana'`, porque JavaScript acepta comillas simples.
- **B.** `JSON.parse()` lanza `SyntaxError`; el módulo lo captura, muestra una advertencia y devuelve el valor por defecto.
- **C.** `JSON.parse()` devuelve `null` sin lanzar error, y la aplicación continúa con ese valor.
- **D.** `getItem()` lanza `SyntaxError` al leer el texto; el módulo propaga la excepción a la aplicación.

15. Un desarrollador ejecuta `localStorage.setItem("config", JSON.stringify(undefined))` y más tarde lee la clave con `JSON.parse()`. ¿Qué ocurre?

- **A.** Se guarda el texto `"null"` y la lectura posterior devuelve `null` sin producir errores.
- **B.** No se guarda ninguna entrada, y `getItem()` devuelve `null` en la lectura posterior.
- **C.** `JSON.stringify()` lanza `TypeError` y la escritura nunca llega a ejecutarse.
- **D.** Se guarda el texto `"undefined"` y la lectura posterior con `JSON.parse()` lanza `SyntaxError`.

16. Se tiene `const usuario = { id: 7, nombre: "Ana", token: "abc123" }` y se necesita que el campo `token` no aparezca en el texto que se guarda. ¿Qué llamada lo logra?

- **A.** `JSON.stringify(usuario, ["token"])`
- **B.** `JSON.stringify(usuario, null, ["token"])`
- **C.** `JSON.stringify(usuario, (k, v) => k === "token" ? undefined : v)`
- **D.** `JSON.stringify(usuario, (k, v) => k === "token" ? null : v)`

17. La versión 1 de una aplicación guardaba `{ nombre: "Ana Pérez" }` y la versión 2 guarda `nombre` y `apellido` por separado. ¿Qué práctica evita que la actualización deje inutilizables los datos de los usuarios existentes?

- **A.** Incluir un número de versión en el dato y escribir una función de migración que valide y convierta el formato antiguo.
- **B.** Ejecutar `localStorage.clear()` en cada despliegue para que todos los usuarios partan siempre de un estado limpio.
- **C.** Confiar en que `JSON.parse()` adaptará la estructura antigua a la que espera el código nuevo.
- **D.** Guardar el dato con sangría mediante el tercer parámetro de `JSON.stringify()` para que sea más legible.

18. Una tienda en línea guarda el carrito en `localStorage` como `[{ idProducto, cantidad, precio }]` y el servidor usa ese precio al cobrar. ¿Cuál es el problema principal de este diseño?

- **A.** Que `JSON.stringify()` no puede representar números decimales, por lo que los precios perderían exactitud.
- **B.** Que `localStorage` no admite arreglos de objetos, así que el carrito debe guardarse únicamente con cookies.
- **C.** Que otros orígenes pueden leer el carrito y modificar sus cantidades desde sus propias páginas.
- **D.** Que el usuario puede editar el valor guardado; solo deben almacenarse identificadores y cantidades, y el servidor recalcula el precio.

19. Una aplicación lee del almacenamiento el texto de una tarea y lo inserta en la página con `innerHTML`. Si el valor guardado contiene `<img src=x onerror=...>`, ¿cuál es el riesgo y cómo se corrige?

- **A.** No hay riesgo, porque la política del mismo origen impide que ese texto ejecute código en la página.
- **B.** Se ejecutaría código en el origen de la aplicación; se corrige insertando el texto con `textContent` o `createElement()`.
- **C.** No hay riesgo, porque `JSON.stringify()` escapa automáticamente las etiquetas HTML al guardar el valor.
- **D.** Solo hay riesgo con `sessionStorage`; se corrige moviendo el dato a `localStorage`, que valida su contenido.

20. El Ejemplo 8 del documento obtiene datos con `fetch()`, los transforma con `map()` y los guarda en `localStorage`. ¿Por qué la unidad afirma que esto no equivale a un proceso ETL completo?

- **A.** Porque `fetch()` no puede extraer datos en formato JSON de una API externa al navegador.
- **B.** Porque `map()` solo reordena los elementos y no puede considerarse una transformación de datos.
- **C.** Porque carece de volumen, orquestación, auditoría y un destino compartido; `localStorage` es una caché privada.
- **D.** Porque un proceso ETL solo puede ejecutarse en el servidor de la aplicación, nunca en herramientas especializadas.

SECCIÓN II. RESPUESTA ABIERTA

Preguntas 21 a 25 · Explique, analice, compare o argumente

21. Compare `localStorage`, `sessionStorage` y las cookies en cuanto a duración, alcance y envío al servidor. Con base en esa comparación, argumente qué mecanismo elegiría para cada uno de estos datos: el tema visual de una aplicación, el borrador de un formulario extenso y el identificador de una sesión autenticada.

Comparación. localStorage: duración indefinida (persiste al cerrar el navegador hasta que se borre); alcance: todo el origen (esquema + host + puerto), compartido entre pestañas y ventanas; no se envía al servidor. **sessionStorage:** dura lo que la pestaña o ventana (sobrevive a las recargas y se descarta al cerrarla); alcance: solo esa pestaña dentro del origen (una pestaña nueva parte vacía); no se envía al servidor. **Cookies:** duración definida con Expires/Max-Age (o hasta cerrar el navegador si no se indica); alcance: dominio y ruta, con atributos Secure, HttpOnly y SameSite; el navegador las envía automáticamente al servidor en cada petición al dominio (capacidad cercana a 4 KB).
Elección. Tema visual: localStorage, porque es una preferencia no sensible que debe recordarse entre visitas y pestañas y que solo usa el cliente, así que no tiene sentido enviarla al servidor en cada petición. **Borrador de un formulario extenso:** sessionStorage, porque es un dato temporal ligado a una sola pestaña: sobrevive a una recarga accidental, no se mezcla con borradores abiertos en otras pestañas, se descarta al terminar y admite más volumen que una cookie sin viajar en cada petición (si hubiera que retomarlo otro día se usaría localStorage). **Identificador de sesión autenticada:** cookie con HttpOnly, Secure y SameSite emitida por el servidor: el navegador la envía sola con cada petición y, al ser HttpOnly, JavaScript (y por tanto un ataque XSS) no puede leerla, algo que no ocurre con Web Storage.

22. Explique qué ocurre, paso a paso, cuando un programa guarda un objeto directamente con `setItem()` y luego intenta leerlo con `JSON.parse()`. Describa cómo se corrige el procedimiento y por qué, tras la corrección, un número recuperado conserva su tipo numérico.

Qué ocurre, paso a paso. (1) Web Storage solo guarda cadenas de texto; con `localStorage.setItem('k', obj)` el navegador convierte el objeto con `String(obj)` y guarda el texto `[object Object]`: las propiedades se pierden y no se produce ningún error en ese momento. (2) `getItem('k')` devuelve esa cadena. (3) `JSON.parse('[object Object]')` lanza **SyntaxError**, porque ese texto no es JSON válido, y el dato original ya no se puede recuperar.
Corrección. Serializar al guardar y deserializar al leer: `localStorage.setItem('k', JSON.stringify(obj))` y `const obj = JSON.parse(localStorage.getItem('k'))`, con try/catch y un valor por defecto si `getItem` devuelve `null` o el JSON es inválido.
Por qué el número conserva su tipo. `JSON.stringify(5)` produce el texto `5` (sin comillas), que Web Storage guarda como cadena. Al leerlo, `JSON.parse('5')` interpreta la sintaxis JSON, donde un literal sin comillas es un número, y devuelve el *number* 5. Un texto se guardaría con comillas (`"5"`) y volvería como *string*. El tipo viaja codificado en la propia sintaxis JSON; sin JSON, `getItem` siempre devolvería la cadena `"5"` y `5 + 1` daría `"51"`.

23. En el módulo `almacenamiento.js` las funciones reciben el nombre del tipo de almacenamiento (`"local"` o `"session"`) y devuelven valores en lugar de propagar excepciones. Analice las razones de estas dos decisiones de diseño y relaciónelas con la degradación elegante.

1) Recibir el tipo ('local' o 'session'). `localStorage` y `sessionStorage` implementan la misma interfaz Storage (getItem, setItem, removeItem). Un único módulo parametrizado con el nombre del tipo evita duplicar código, centraliza en un solo lugar la comprobación de disponibilidad, el try/catch, el JSON y las claves con prefijo, permite cambiar de mecanismo con solo cambiar un argumento y facilita probar. Además, resolver `window[tipo]` dentro del `try` protege incluso la simple lectura, que en algunos navegadores lanza SecurityError cuando el acceso está bloqueado.
2) Devolver valores en lugar de propagar excepciones. El almacenamiento puede fallar por causas ajenas al programa: acceso bloqueado o modo privado (SecurityError), cuota agotada (QuotaExceededError) o contenido manipulado o corrupto (SyntaxError al hacer parse). Como la persistencia es una mejora y no el corazón de la aplicación, una excepción no capturada interrumpiría el flujo y rompería la página. Si las funciones devuelven un valor (true/false al guardar, el valor por defecto al leer), quien las llama decide qué hacer.
Relación con la degradación elegante. La aplicación sigue funcionando con su estado en memoria cuando el almacenamiento no está disponible: solo pierde la persistencia (y, si afecta al usuario, se le informa), en lugar de dejar de funcionar por completo.

24. Elija tres tipos de valor que no sobreviven correctamente a `JSON.stringify()` y `JSON.parse()`. Para cada uno, explique qué ocurre con él y cómo se resuelve, y argumente por qué la omisión silenciosa de datos es el comportamiento más peligroso.

1) Date. `JSON.stringify` lo convierte en una cadena ISO 8601 (mediante toJSON) y `JSON.parse` la devuelve como *string*, por lo que `vence.getFullYear()` lanza TypeError. Solución: usar un *reviver* que reconstruya `new Date(valor)` (validando que la fecha sea válida).
2) Set (y Map). Se serializan como `{}`: el contenido se pierde y no hay error. Solución: usar un *replacer* que los convierta en arreglo (`[...set]`, o `[...map]` para Map) y un *reviver* o una reconstrucción explícita que los vuelva a crear (`new Set(arr)`, `new Map(arr)`).
3) undefined (también funciones y Symbol). En un objeto la propiedad simplemente desaparece; en un arreglo se convierte en `null`; y `JSON.stringify(undefined)` devuelve `undefined`, de modo que `setItem` guarda la cadena `'undefined'` y el posterior `JSON.parse` lanza SyntaxError. Solución: usar `null` explícito para «sin valor», normalizar antes de guardar, asignar valores por defecto al leer y no guardar funciones, sino datos desde los que se reconstruya el comportamiento. (Algo parecido ocurre con `NaN` e `Infinity`, que pasan a `null`.)
Por qué la omisión silenciosa es lo más peligroso. Un error con excepción (por ejemplo, BigInt lanza TypeError) se detecta de inmediato y obliga a corregir. En cambio, cuando el dato se omite o se transforma sin avisar, la aplicación cree que guardó bien, no hay ningún mensaje y el fallo aparece más tarde, lejos de su causa, como un dato faltante, un valor null o una lógica incorrecta, difícil de depurar y capaz de corromper decisiones o cálculos sin que nadie lo note. Por eso conviene validar al leer, probar el ciclo completo guardar-leer y versionar el formato.

25. Argumente por qué no deben guardarse contraseñas ni tokens de larga duración en Web Storage. En su respuesta considere quién puede leer esos datos, qué papel cumple el cross-site scripting (XSS) y qué alternativa recomienda la unidad para las sesiones autenticadas.

Quién puede leer los datos. Web Storage es legible por cualquier JavaScript que se ejecute en el origen: el código propio, los scripts de terceros incluidos (CDN, analítica, widgets), las extensiones del navegador y cualquier persona con acceso al equipo mediante las herramientas de desarrollo. No tiene el atributo HttpOnly, no se cifra y localStorage no caduca, de modo que un secreto guardado ahí permanece disponible indefinidamente.
Papel del XSS. La política del mismo origen impide que *otros* orígenes lean el almacenamiento, pero no protege contra código malicioso que se ejecuta *dentro* del origen. Si existe una vulnerabilidad XSS (por ejemplo, insertar con innerHTML un texto no confiable), el atacante ejecuta `localStorage.getItem('token')` y envía el valor a su servidor con fetch. Con un token de larga duración puede suplantar al usuario durante mucho tiempo, incluso después de que se corrija la falla. Una contraseña guardada en texto plano, además, expone la cuenta y todas aquellas en las que el usuario la reutilice.
Alternativa recomendada por la unidad. Mantener la sesión autenticada en una **cookie emitida por el servidor con HttpOnly, Secure y SameSite**, con caducidad corta y renovación controlada por el servidor. JavaScript no puede leer una cookie HttpOnly, de modo que un XSS no puede robar el identificador (aunque la aplicación debe seguir evitando el XSS: escapar datos y usar textContent o createElement en lugar de innerHTML), y el navegador la envía automáticamente. Las contraseñas nunca se guardan en el navegador; en Web Storage solo se dejan datos no sensibles, como preferencias o cachés.

SECCIÓN III. APLICACIÓN MEDIANTE EJEMPLOS

Preguntas 26 a 30 · Proponga ejemplos aplicados a situaciones reales

26. Una empresa de comercio electrónico quiere recordar el idioma elegido por el cliente y también el paso actual de un asistente de compra de varios pasos. Proponga un ejemplo para cada dato indicando el mecanismo de almacenamiento, la clave con prefijo y versión, el objeto JSON que guardaría y la razón de su elección.

Idioma elegido: localStorage. Clave: `tienda:v1:idioma`. Valor: `{"version":1,"codigo":"es-CO"}`. Razón: es una preferencia no sensible que debe sobrevivir al cierre del navegador y compartirse entre pestañas; solo la usa el cliente. (Si además el servidor necesitara el idioma en cada petición sin programar envíos manuales, se usaría una cookie.)
Paso actual del asistente de compra: sessionStorage. Clave: `tienda:v1:asistente`. Valor: `{"version":1,"paso":3,"totalPasos":5,"envio":"estandar"}` (sin datos de pago ni otra información sensible). Razón: es un estado temporal ligado a una compra en una pestaña concreta; sobrevive a una recarga accidental, no se mezcla con otras compras abiertas en otras pestañas y se descarta al cerrar la pestaña, sin dejar rastros. Se escribiría con `sessionStorage.setItem('tienda:v1:asistente', JSON.stringify(estado))` y se leería con valor por defecto `{version:1, paso:1}` si no existe.

27. Una organización tiene un panel administrativo con tablas extensas. Diseñe un ejemplo de objeto JSON único para guardar preferencias de interfaz (tema, filas por página y columnas visibles) y describa cómo lo cargaría con valores por defecto, qué validaría de cada propiedad y cómo manejaría un cambio futuro de formato.

Objeto único (clave `panel:v2:preferencias`):
`{"version":2,"tema":"oscuro","filasPorPagina":25,"columnas":["id","nombre","estado","fecha"]}`.
Carga con valores por defecto. Se define una constante `POR_DEFECTO = {version:2, tema:"claro", filasPorPagina:10, columnas:["id","nombre","estado"]}`. Se lee con `try { JSON.parse(localStorage.getItem(clave)) } catch` y, si devuelve null, no es un objeto o el JSON está corrupto, se usa una copia de POR_DEFECTO. Si es válido, se construye un objeto nuevo propiedad por propiedad, usando el valor por defecto solo en la que falle.
Validación. *tema*: debe ser uno de los valores permitidos ('claro' u 'oscuro'). *filasPorPagina*: entero (`Number.isInteger`) dentro de una lista permitida (10, 25, 50, 100). *columnas*: arreglo de cadenas, filtrado contra las columnas que existen realmente y no vacío. *version*: número conocido.
Cambio futuro de formato. El campo `version` (y la versión en la clave) permite escribir una función `migrar(datos)` que convierta el formato antiguo al nuevo (p. ej., v1 usaba `filas` y v2 usa `filasPorPagina`), valide el resultado, lo vuelva a guardar y, si la versión es desconocida, vuelva a los valores por defecto sin romper la aplicación. Código completo en el Anexo (página 11).

28. Una empresa de logística consulta una API cada vez que se abre una pantalla, aunque sus datos cambian poco. Proponga una política de caché en `localStorage`: estructura JSON con marca de tiempo, tiempo de vigencia (TTL), qué hacer si la caché venció y qué hacer si no hay conexión.

Estructura (clave `logistica:v1:cache:envios`):
`{"version":1,"guardadoEn":1730800000000,"ttlMs":300000,"datos":[ ... ]}`, donde `guardadoEn` es la marca de tiempo (`Date.now()` en milisegundos) y `ttlMs` el tiempo de vigencia (p. ej., 5 minutos, ajustable según la frecuencia de cambio de los datos).
Política. (1) Al abrir la pantalla se lee la caché con try/catch y se valida su forma. (2) Si existe y `Date.now() - guardadoEn < ttlMs`, está **vigente**: se usa sin llamar a la API. (3) Si **venció** (o no existe), se consulta la API con fetch; si responde bien, se reemplaza la caché con datos nuevos y nueva marca de tiempo. (4) Si **no hay conexión** (`navigator.onLine` es false o fetch falla): si hay caché vencida se muestra como último dato conocido, con un aviso («datos de hace X minutos») y se reintenta al volver la red; si no hay caché, se muestra un mensaje de error con opción de reintentar. (5) Si el guardado lanza QuotaExceededError, se captura y la aplicación continúa sin caché. Código completo en el Anexo (página 11).

29. Un sistema de información de inventarios guarda productos con fecha de vencimiento (`Date`), un conjunto de etiquetas (`Set`) y un campo interno de costo que no debe quedar en el navegador. Proponga cómo serializaría y reconstruiría estos datos, indicando el uso de `replacer` y `reviver` en su ejemplo.

Ejemplo: `{id:1, nombre:"Leche", vence:new Date(...), etiquetas:new Set(["lacteo","frio"]), costo:1200}`.
Serializar. Un *replacer* pasado como segundo argumento de `JSON.stringify` omite `costo` (devuelve `undefined` para esa clave, así no llega al navegador; lo ideal es que el servidor ni siquiera lo envíe) y convierte el Set en arreglo (`valor instanceof Set ? [...valor] : valor`). El Date se guarda solo como cadena ISO 8601 gracias a toJSON. Se guarda:
`{"id":1,"nombre":"Leche","vence":"2026-12-31T00:00:00.000Z","etiquetas":["lacteo","frio"]}`.
Reconstruir. Un *reviver* pasado como segundo argumento de `JSON.parse`: si la clave es `vence` devuelve `new Date(valor)` (verificando que sea una fecha válida) y si la clave es `etiquetas` devuelve `new Set(valor)`. Así `leido.vence.getFullYear()` y `leido.etiquetas.has('frio')` funcionan. Código completo en el Anexo (página 11).

30. Una empresa de analítica de datos opera un proceso ETL especializado y también ofrece un panel web a sus clientes. Proponga un ejemplo concreto de cómo participa el navegador en el flujo de datos (entrada, consumo y presentación) y qué tareas deben permanecer en el servidor o en el ETL, explicando por qué.

Ejemplo: panel web de ventas de una empresa de analítica.
Entrada (navegador): el cliente elige filtros (rango de fechas, región, métrica) y preferencias (tema, columnas visibles); el navegador las valida y las guarda en localStorage (clave `analitica:v1:filtros`) para recordarlas en la próxima visita.
Consumo (navegador): con esos filtros el panel hace `fetch('/api/ventas?...')` a una API del servidor que entrega datos ya procesados por el ETL (agregados, en JSON). Puede guardarlos en localStorage como caché privada con marca de tiempo y TTL para no repetir la consulta cuando los datos cambian poco, y manejar la falta de conexión mostrando el último dato conocido.
Presentación (navegador): transforma con map/filter/sort lo mínimo necesario para la vista (formatear fechas y monedas, ordenar, paginar, graficar) y lo inserta con textContent o createElement, nunca con innerHTML sin sanear.
Debe permanecer en el servidor o en el ETL: la extracción desde múltiples fuentes, el manejo de grandes volúmenes, la limpieza, las validaciones de calidad, las uniones y agregaciones pesadas, la orquestación y programación de procesos, la auditoría y trazabilidad, la carga en un destino compartido (almacén de datos) y la autorización (qué datos puede ver cada cliente), además de cualquier cálculo oficial (KPI, precios, costos).
Por qué: el navegador es un entorno no confiable (el usuario puede editar lo que guarda), con cuota limitada, hilo principal que se bloquea con cargas grandes y almacenamiento privado por dispositivo, sin garantía de persistencia, sin auditoría ni orquestación y sin un destino compartido. Por eso solo actúa como una caché y una interfaz, y no como un ETL completo.

ANEXO · Código de apoyo (preguntas 27, 28 y 29)

Estudiantes: Kevin Santiago Urán, Eliseth Nathaly Velasco Gómez, Juan Manuel Orozco y Mariana Diaz Jaramillo.

Pregunta 27 · Preferencias de interfaz con valores por defecto, validación y migración

```
const CLAVE = "panel:v2:preferencias";
const POR_DEFECTO = { version: 2, tema: "claro", filasPorPagina: 10,
                      columnas: ["id", "nombre", "estado"] };
const TEMAS = ["claro", "oscuro"];
const FILAS = [10, 25, 50, 100];
const COLUMNAS = ["id", "nombre", "estado", "fecha", "responsable"];

function migrar(d) {                       // v1 usaba "filas"; v2 usa "filasPorPagina"
  if (d && d.version === 1) {
    d = { ...d, filasPorPagina: d.filas, version: 2 };
    delete d.filas;
  }
  return d;
}

function cargarPreferencias() {
  let crudo = null;
  try { crudo = JSON.parse(localStorage.getItem(CLAVE)); }
  catch (e) { crudo = null; }              // JSON corrupto o almacenamiento bloqueado
  if (!crudo || typeof crudo !== "object" || Array.isArray(crudo))
    return structuredClone(POR_DEFECTO);   // primera visita (null) o forma invalida
  const d = migrar(crudo);
  if (d.version !== 2) return structuredClone(POR_DEFECTO);   // version desconocida
  const cols = Array.isArray(d.columnas) ? d.columnas.filter(c => COLUMNAS.includes(c)) : [];
  return {
    version: 2,
    tema: TEMAS.includes(d.tema) ? d.tema : POR_DEFECTO.tema,
    filasPorPagina: FILAS.includes(d.filasPorPagina) ? d.filasPorPagina : POR_DEFECTO.filasPorPagina,
    columnas: cols.length ? cols : [...POR_DEFECTO.columnas]
  };
}

function guardarPreferencias(p) {
  try { localStorage.setItem(CLAVE, JSON.stringify(p)); return true; }
  catch (e) { return false; }              // cuota agotada o acceso bloqueado
}
```

Pregunta 28 · Caché con marca de tiempo y TTL

```
const CLAVE_CACHE = "logistica:v1:cache:envios";
const TTL_MS = 5 * 60 * 1000;              // 5 minutos

function leerCache() {
  try {
    const c = JSON.parse(localStorage.getItem(CLAVE_CACHE));
    if (c && typeof c.guardadoEn === "number" && Array.isArray(c.datos)) return c;
  } catch (e) { /* cache ilegible: se ignora */ }
  return null;
}

function guardarCache(datos) {
  try {
    localStorage.setItem(CLAVE_CACHE, JSON.stringify(
      { version: 1, guardadoEn: Date.now(), ttlMs: TTL_MS, datos }));
  } catch (e) { /* QuotaExceededError: se sigue sin cache */ }
}

async function obtenerEnvios() {
  const c = leerCache();
  if (c && Date.now() - c.guardadoEn < (c.ttlMs ?? TTL_MS))
    return { datos: c.datos, origen: "cache" };                 // vigente
  try {
    const r = await fetch("/api/envios");
    if (!r.ok) throw new Error("HTTP " + r.status);
    const datos = await r.json();
    guardarCache(datos);                                        // renueva marca y TTL
    return { datos, origen: "red" };
  } catch (e) {
    if (c) return { datos: c.datos, origen: "cache-vencida" };  // sin conexion: ultimo dato + aviso
    throw e;                                                    // sin cache: error y reintento
  }
}
```

Pregunta 29 · Replacer y reviver (Date, Set y campo interno «costo»)

```
const producto = {
  id: 1, nombre: "Leche",
  vence: new Date("2026-12-31T00:00:00Z"),
  etiquetas: new Set(["lacteo", "frio"]),
  costo: 1200
};

const replacer = (clave, valor) => {
  if (clave === "costo") return undefined;        // se omite: no queda en el navegador
  if (valor instanceof Set) return [...valor];    // Set -> arreglo
  return valor;                                   // Date ya llega como cadena ISO (toJSON)
};

localStorage.setItem("inventario:v1:producto:1", JSON.stringify(producto, replacer));
// {"id":1,"nombre":"Leche","vence":"2026-12-31T00:00:00.000Z","etiquetas":["lacteo","frio"]}

const reviver = (clave, valor) => {
  if (clave === "vence" && typeof valor === "string") {
    const f = new Date(valor);
    return Number.isNaN(f.getTime()) ? null : f;  // valida la fecha
  }
  if (clave === "etiquetas" && Array.isArray(valor)) return new Set(valor);
  return valor;
};

const leido = JSON.parse(localStorage.getItem("inventario:v1:producto:1"), reviver);
leido.vence.getFullYear();     // 2026
leido.etiquetas.has("frio");   // true
```